

Ludex: A Machine-Learning Based Hybrid Game Recommendation System

A Project Report submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Aditya Apoorva Negi (112315010)

Aditya Harikrishnan (112315011)

M. Arsalan Siddiqui (112315108)

Jonathan Jesni (112315078)

Om Arun Thombre (112315119)

Under the Supervision of

Dr. Mohammad Aklaqur Rahman

Semester: V



Department of Computer Science and Engineering
Indian Institute of Information Technology, Pune
(An Institute of National Importance by an Act of Parliament)
December 2025

BONAFIDE CERTIFICATE

This is to certify that the project report entitled “**Ludex: A Machine-Learning Based Hybrid Game Recommendation System**” submitted by **Aditya Apoorva Negi** bearing the MIS No: **112315010**, **Aditya Harikrishnan** bearing the MIS No: **112315011**, **M. Arsalan Siddiqui** bearing the MIS No: **112315108**, **Jonthan Jesni** bearing the MIS No: **112315078**, and **Om Arun Thombre** bearing the MIS No: **112315119** in completion of their project work under the guidance of **Dr. Mohammad Aklaqur Rahman** is accepted for the project report submission in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in the **Department of Computer Science and Engineering**, Indian Institute of Information Technology, Pune (IIIT Pune), during the academic year **2025-26**.

Dr. Mohammad Aklaqur Rahman
Singh

Project Supervisor
Adjunct Assistant Professor
Department of ECE
IIIT Pune

Dr. Bhupendra

Head of the Department
Assistant Professor
Department of CSE
IIIT Pune

Project Viva-voce held on: 1st December, 2025

Undertaking for Plagiarism

We **Aditya Apoorva Negi, Aditya Harikrishnan, M. Arsalan Siddiqui, Jonathan Jesni, and Om Arun Thombre** solemnly declare that the research work presented in the **report/dissertation** titled “**Ludex: A Machine-Learning Based Hybrid Game Recommendation System**” is solely **our** research work with no significant contribution from any other person. Any small contribution/help wherever taken has been duly acknowledged and the complete report has been written by **us**. We understand the zero-tolerance policy of **Indian Institute of Information Technology, Pune** towards plagiarism. Therefore, **we** declare that no portion of our **report** has been plagiarized and any material used as reference is properly referred/cited. We undertake that if we are found guilty of any formal plagiarism in the above titled thesis even after the awarding of the degree, the Institute reserves the right to withdraw/revoke our **B. Tech** degree.

Student's Name and Signature with Date

Aditya Apoorva Negi

Aditya Harikrishnan

M. Arsalan Siddiqui

Jonathan Jesni

Om Arun Thombre

Conflict of Interest

Manuscript title: Ludex: A Machine-Learning Based Hybrid Game Recommendation System

The authors whose names are listed immediately below certify that they have NO affiliations with or involvement in any organization or entity with any financial interest (such as honoraria; educational grants; participation in speakers' bureaus; membership, employment, consultancies, stock ownership, or other equity interest; and expert testimony or patent-licensing arrangements), or non-financial interest (such as personal or professional relationships, affiliations, knowledge or beliefs) in the subject matter or materials discussed in this manuscript.

Student's Name and Signature with Date

Aditya Apoorva Negi

Aditya Harikrishnan

M. Arsalan Siddiqui

Jonathan Jesni

Om Arun Thombre

ACKNOWLEDGEMENT

This project would not have been possible without the help and cooperation of many people. We would like to thank the people who helped us directly or indirectly in the completion of this project.

First and foremost, we would like to express our gratitude to our honorable Director, **Prof. Shireesh B. Kedare**, for providing his kind support in various aspects.

We would like to express our gratitude to our project guide **Dr. Mohammad Aklaqur Rahman, Department of ECE**, for providing excellent guidance, encouragement, inspiration, constant and timely support throughout this **B. Tech Project**.

We would like to express our gratitude to **Dr. Bhupendra Singh, Head of Department, Department of CSE**, for providing his kind support in various aspects.

We would also like to thank all the faculty members in the **Department of CSE and Department of ECE** and our fellow classmates for their steadfast and strong support and engagement with this project.

Abstract

The rapid growth of the gaming industry and online gaming platforms in recent times, mainly Steam, has created a growing need for highly personalized and accurate game recommendations to be available on the platforms. As the number of available titles continues to grow, players face increasing difficulty in discovering games that align with their preferences and interests. Traditional recommendation approaches, such as pure Content-Based Filtering (CBF) or pure Collaborative Filtering (CF), suffer from well-known limitations such as cold-start issues, sparse user–item interaction matrices, and limited personalization when used in isolation.

To tackle these issues, this project presents **Ludex**, a hybrid, machine-learning recommendation system that integrates a fusion of CBF and CF within a single pipeline. The system incorporates a rich CBF module built using **TF–IDF** based content embeddings, developer/publisher **one-hot encoding**, and **anchor-weighted** user profiling. The accompanying CF module is implemented using **implicit-feedback Alternating Least Squares (ALS)** trained on a large-scale user data set gathered from **Steam Web API**. To improve robustness, the CF pipeline includes popularity-adjusted scoring and **automated user-enrichment** to mitigate missing-user issues.

A key contribution of this project is a **two-stage hybrid architecture**:

1. **Independent candidate generation** from CBF and CF, each producing its own top-K ranked list.
2. **Union-based score fusion**, where raw CBF and CF scores are **separately normalized**, blended through a **tuneable α -weight hybrid function**, and then passed through a single **Maximal Marginal Relevance (MMR)** re-ranking step on the hybrid scores.

In addition, the system includes **cold-start fallbacks**, automatically switching to pure CF recommendations when CBF cannot be computed (e.g., no content anchors, missing catalogue entries, or Steam API failures).

Experimental analysis demonstrates that this hybrid pipeline produces more diverse, relevant, and personalized recommendations than pure CBF or pure CF, while also being resilient to sparse data and cold-starts.

Future work may explore Transformer-based game embeddings, graph neural network models for friend-network augmentation, and reinforcement-learning–driven ranking strategies to further improve personalization and long-term engagement.

Keywords: Hybrid Recommendation System, Machine Learning, TF–IDF, Implicit ALS, MMR, Steam API.

TABLE OF CONTENTS

BONAFIDE CERTIFICATE.....	i
Undertaking for Plagiarism.....	ii
Conflict of Interest.....	iii
ACKNOWLEDGEMENT.....	iv
Abstract.....	v
TABLE OF CONTENTS.....	vi
List of Nomenclature.....	viii
List of Formulae.....	ix
List of Figures.....	x
List of Tables.....	xi
Introduction.....	1
1.1 Overview of Work.....	1
1.2 Literature Review.....	3
1.3 Motivation of the Work.....	4
1.4 Research Gap.....	5
Problem Statement.....	7
2.1 Research Objectives.....	8
2.2 Analysis and Design.....	9
2.2.1 System Requirement Analysis.....	9
2.2.2 Architectural Design Principles.....	9
2.2.3 High Level System Architecture.....	11
Proposed Work.....	13
3.1 Methodology of Work.....	13
3.1.1 Data Acquisition and Processing.....	13
3.1.2 Feature Engineering and CBF Pipeline.....	13
3.1.3 User Profiling through Anchor-Based Preference Modelling.....	14
3.1.4 Collaborative Filtering Using Implicit ALS.....	14
3.1.5 Hybrid Score Fusion.....	15
3.1.6 Diversity-Aware Re-Ranking using MMR.....	15
3.1.7 Cold Start and Failure Handling.....	16
3.1.8 Caching, Persistence, and System Optimisation.....	16
3.1.9 Complete Pipeline.....	16
3.1.10 System Components and Implementation Modules.....	17
3.2 Dataset Description.....	18
1. Steam Friends Network Dataset (friends.json).....	18
2. Steam Owned Games Dataset (owned.json).....	19
3. Seed SteamID List (seed_steamids.txt).....	19
4. Steam Game Metadata Dataset (game_details.csv).....	19
Results and Discussion.....	21
4.1 Quantitative Results.....	21

4.2 Discussion of Results.....	22
4.3 Why the Hybrid Model Performs Best Overall.....	23
4.4 Summary of Findings.....	23
4.5 Practical Implementation and Deployment.....	24
Conclusion and Future Scope.....	26
Conclusion.....	26
Future Scope.....	26
Overall Outlook.....	28
References.....	29

List of Nomenclature

Content-Based Filtering(CBF)

A recommendation technique that uses item(game) features (genres, tags, descriptions, developers, etc.) to match users with similar content profiles.

CBF relies heavily on TF-IDF vectors and the quality of the available metadata.

Long-Tailed Data

Long-tailed data in recommendation systems refers to a distribution where a few popular items ("head items") get most of the interactions, while a large number of niche or less-popular items ("long-tail items") have very few interactions.

Collaborative Filtering(CF)

A recommendation method that uses user-to-user or item-to-item similarity patterns based on behavior rather than metadata.

CF learns from patterns such as “Users who played X also enjoyed Y.”

TF-IDF — Term Frequency–Inverse Document Frequency

A text-weighting technique used to convert metadata (tags, genres, descriptions) into numerical vectors. Words/tags that are common across many games get lower weight; unique or meaningful terms get higher weight.

OHE — One-Hot Encoding

A method of encoding categorical game metadata (developer, publisher) into binary vectors.

Implicit Feedback

A form of user preference signal that is **inferred from behavior**, not explicitly provided as a rating.

Normalized Playtime Data

Playtime from Steam is raw and highly skewed (some users have 5 hours on a game, others have 5,000). To fix this, the system converts raw minutes into **normalized weights**.

ALS — Alternating Least Squares

A matrix factorization algorithm used in Collaborative Filtering. ALS breaks the user–item matrix into two latent matrices: user factors (U) and item factors (V), and then alternates between optimizing each one to minimize error. Used for large sparse matrices typical of recommendation systems.

MMR — Maximal Marginal Relevance

An algorithm used to re-rank recommendations by balancing relevance (similarity to the user profile) and diversity (reduction of redundancy).

MMR helps avoid giving the user multiple similar games and ensures a varied, balanced recommendation list.

List of Formulae

TF

$$TF(t, d) = f_{t,d} / \sum_{t' \in d} f_{t',d}$$

IDF

$$IDF(t) = \log \left(\frac{N}{1 + df(t)} \right)$$

TF-IDF

$$TFIDF(t, d) = TF(t, d) \cdot IDF(t)$$

COSINE SIMILARITY

$$CosineSim(A, B) = \frac{\sum_i A_i B_i}{\sqrt{\sum_i A_i^2 \cdot \sum_i B_i^2}}$$

NDCG@K

$$DCG@k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i+1)}$$

$$IDCG@k = \sum_{i=1}^k \frac{2^{rel_i^*} - 1}{\log_2(i+1)}$$

$$NDCG@k = \frac{DCG@k}{IDCG@k}$$

Precision@K

$$Precision@k = \frac{\sum_{i=1}^k rel_i}{k}$$

Novelty@K

$$Novelty@k = \frac{1}{k} \sum_{i=1}^k -\log_2(pop_i)$$

Maximal Marginal Relevance

$$MMR(d_j) = \lambda \cdot Rel(d_j) - (1 - \lambda) \max_{d_k \in S} Sim(d_j, d_k)$$

Hybrid Scoring

$$Hybrid(u, i) = \alpha s_{CF}(u, i) + (1 - \alpha) \hat{s}_{CBF}(u, i)$$

Alternate Least Squares

$$ALS x_u = (Y^T C_u Y + \lambda I)^{-1} Y^T C_u p_u$$

Min-Max Normalization

$$MinMax \hat{s} = \frac{s - s_{min}}{s_{max} - s_{min}}$$

List of Figures

Figure 2.1: ALS Matrix Factorization

Figure 2.2: CBF User Anchor Profiling

Figure 2.3: Hybrid Scoring Fusion Process

Figure 2.4: System Diagram

Figure 3.1: Ludex Pipeline

Figure 4.1: Working of the deployed Ludex website

List of Tables

Table 1.1: Summary of Key Literature Reviewed in Recommender Systems

Table 3.1: TF-IDF and OHE Feature Hyperparameters used for CBF

Table 3.2: ALS Hyperparameters for Collaborative Filtering, and Playtime Normalisation and Confidence Transformation Rules

Table 3.3: MMR Re-ranking Parameters and Descriptions, and Hybrid Pipeline Candidate and Output Configuration

Table 3.4: Dataset Overview for Ludex Recommender System

Table 4.1: Average Performance of Various Models across Several Users

Chapter 1

Introduction

1.1 Overview of Work

The rapid growth of Steam as one of the world’s largest PC gaming platforms has created an enormous digital marketplace, hosting hundreds of thousands of titles and attracting over 132 million monthly active users and 41 million concurrent users at peak engagement levels [1]. Due to this scale of the number of choices, players often struggle to discover games that genuinely match their interests. Recent surveys show that over 30% of Steam library games remain unplayed, with the median user leaving more than half of their purchased titles untouched [2]. This widespread issue is closely linked to Steam’s existing recommendation system, which relies heavily on popularity-driven heuristics and engagement metrics. Such mechanisms frequently surface already-successful games while overlooking niche, high-quality, or personalised options, thereby limiting diversity and degrading the user discovery experience [3][4][5]. This popularity-first pipeline also poses significant consequences for independent game creators, who often lack marketing budgets, remain invisible within Steam’s opaque curation system. As highlighted by industry analyses, even strong or innovative indie titles can fail commercially if they do not gain visibility within the platform’s narrow exposure window[6].

Given these challenges, relying solely on either content-based or collaborative filtering is insufficient as each method captures only one side of user preference. A hybrid approach becomes necessary to combine semantic understanding of games with behavioural patterns derived from real player activity.

To address these limitations, this project proposes **Ludex**, a hybrid Machine-Learning based recommendation system integrating data-driven Content-Based Filtering (CBF) and Collaborative Filtering (CF) within a unified and modular architecture. Ludex aims to generate accurate and varied recommendations by using both the descriptive features of games and the playing habits of users.

1. The **Content-Based Filtering (CBF)** engine uses a large TF–IDF feature matrix built from game descriptions, genres, and titles, supplemented with one-hot encoded developer and publisher information. These features form a unified high-dimensional representation of each game. To build a personalised user profile, the system identifies the titles on which the user has spent at least 60 minutes of meaningful playtime (anchor games). The feature vectors of these “anchors” are averaged to form a representative user embedding vector of the user’s taste in games. Using this user vector, the CBF module retrieves a clean list of candidate games based on cosine similarity.
2. The **Collaborative Filtering (CF)** engine is powered by an implicit-feedback Alternating Least Squares (ALS) model trained on player–game interaction data. All interactions are converted into a playtime-normalised confidence score, allowing the

model to learn meaningful behavioural preferences. The pipeline also performs automated user enrichment, meaning that whenever a new or previously unseen user appears, their game data is added to the interactions file and the ALS model is retrained only when necessary. This ensures the CF model always has the user represented in its learned embedding space without requiring full manual updates. To counter popularity bias, raw ALS scores are adjusted using a normalised popularity penalty, enabling fairer visibility for lesser-known titles. Using this refined signal, the CF engine independently generates a behaviour-driven top-K candidate list for each user.

3. To unify the results of these two models, the union set of the top-K raw CBF and CF scores are individually normalised and blended using a tunable α -weight to produce a single hybrid relevance score that balances semantic similarity with behavioural patterns. A final **Maximal Marginal Relevance (MMR)** step is incorporated to improve variety and reduce redundancy, ensuring that recommendations are not only relevant but also diverse.
4. To ensure robustness, the system includes **cold-start and failure-handling** mechanisms. It automatically switches to pure CF when CBF cannot be computed (e.g., missing metadata or no anchor games), and to pure CBF when CF data or models are unavailable. The hybrid logic gracefully handles partial or sparse candidate pools.

All components such as web crawling, preprocessing, TF-IDF feature construction, ALS training, hybrid scoring, and MMR re-ranking are integrated into a single, executable Python pipeline capable of generating recommendations for any Steam user via a command-line interface. The system is implemented using technologies such as **Pandas, NumPy, SciPy, Scikit-Learn, Implicit ALS, Requests, BeautifulSoup**, and **Pickle** for data handling, modelling, API access, and persistence. The result is a scalable, modular, and reproducible framework designed to deliver personalised and diverse game recommendations in real time.

Given this context, it is essential to examine existing research in recommender system methodologies to understand their strengths and limitations. The following literature review summarizes key studies forming the foundation of Ludex.

1.2 Literature Review

A substantial body of research exists on recommender systems, particularly in the domains of Content-Based Filtering (CBF), and Collaborative Filtering (CF). Table 1 summarizes the key studies and insights relevant to this project.

Table 1.1: Summary of Key Literature Reviewed in Recommender Systems

<u>Study</u>	<u>Method Used</u>	<u>Key Findings/Takeaways</u>
Breese et al. (1998)[7]; Su & Khoshgoftaar (2009)[8]; Koren et al. (2009)[9]	Memory-based CF and matrix-factorisation (ALS/MF)	Demonstrated that CF uncovers latent preferences but suffers from data sparsity, scalability issues, and cold-start, motivating model-based, regularised MF approaches.
Adomavicius & Tuzhilin (2005)[10]	Survey of recommender system paradigms	Formalised strengths and weaknesses of CBF, CF, and hybrid methods; highlighted the need for approaches that address overspecialisation and cold-start.
Lops et al. (2011)[11]	Survey of CBF models using TF-IDF and feature embeddings	CBF is effective and explainable for new items but heavily depends on metadata quality and often over-specialises, reducing diversity.
Fernández-Tobías et al. (2019)[12]	Comparative review of cold-start solution families (active learning, semantic features, cross-domain)	Classified effective strategies for new-user and new-item cold start. Emphasised semantic attribute modelling (TF-IDF) for new items and active-learning preference elicitation for new users.
Zhou et al. (2023)[13]	Comprehensive survey of hybrid recommenders	Showed that hybrid architectures (weighted, cascade, switching) generally outperform standalone CBF/CF and benefit from rich metadata and representation learning.

The literature highlights complementary strengths and limitations of CBF and CF. CBF offers explainability and strong new-item handling, whereas CF captures latent behavioural preferences but suffers from sparsity and cold-start issues. Hybridisation is widely recommended, yet practical implementations tailored to gaming environments remain

underdeveloped, particularly concerning metadata richness, behavioural variability, and diversity preservation.

1.3 Motivation of the Work

Steam’s vast digital ecosystem has created unprecedented opportunities for players and game developers, but it has also introduced several challenges that directly affect the quality of user experience and the visibility of emerging titles. The shortcomings of Steam’s popularity-driven recommendation pipeline such as poor personalisation, limited diversity, and an entrenched bias toward already-successful games highlight the need for a more balanced and user-centric solution[1]-[4],[6]. These observations form the key motivations behind this project:

- **Improving Personalized Game Discovery**

Steam’s default algorithm frequently recycles mainstream or already-successful titles, resulting in poor personalization and reduced exposure to high-quality niche games [2]–[4]. This problem is reflected in the rising share of unplayed purchases and the limited visibility of relevant long-tail content [3], [6]. A hybrid ML-based recommender that integrates semantic and behavioural signals can better capture user preferences and surface games that genuinely match individual interests.

- **Supporting Independent and Niche Game Developers**

Discoverability studies show that indie games face significant visibility barriers due to Steam’s popularity-driven ranking mechanisms and early-momentum bias [2], [3], [6]. Algorithms that heavily weight global interaction statistics systematically favour established hits over emerging titles. By shifting the ranking signal toward relevance rather than popularity, systems like Ludex can mitigate this imbalance and provide smaller studios with a fairer opportunity to reach their intended audience.

- **Enhancing the Overall Gaming Experience of Users**

Academic and industry literature emphasises that diverse and personalised recommendation lists increase user satisfaction and long-term engagement [5], [10], [11]. By exposing players to a broader range of genres, mechanics, and experiences, a hybrid recommender can enhance the depth of gameplay discovery while preserving relevance. This leads to richer engagement patterns and healthier ecosystem dynamics for both players and developers.

- **Addressing Personal Dissatisfaction with Existing Recommenders**

Research on collaborative filtering techniques shows that popularity bias and overspecialisation are inherent weaknesses of many large-scale online recommenders [8]–[10], [12], [13]. These issues are apparent in Steam’s behaviour, where the recommender often repeats predictable AAA or free-to-play staples rather than exploring the user’s nuanced interests. This project is partly motivated by firsthand frustration with such limitations, reinforcing the need for a hybrid architecture that corrects these shortcomings.

- **Need for a Balanced, and Transparent Recommendation Framework**

Despite extensive research in CBF and CF methods [5], [7]–[11], existing commercial implementations often lack transparency or simplicity. Designing a hybrid system like Ludex provides the opportunity to integrate well-established recommendation principles into a clean, extensible model suitable for large catalogues and diverse user

profiles. Such an approach aligns with findings from deep-learning-based recommender surveys [12] and hybrid system analyses [13], which highlight the importance of combining content and behavioural signals.

Together, these motivations justify the development of Ludex as a balanced, user-centric, and technically robust recommendation system designed specifically for large-scale game ecosystems.

1.4 Research Gap

Extensive research exists in the domains of content-based filtering (CBF), collaborative filtering (CF), and hybrid recommendation systems [5], [7]–[13]. However, several critical limitations remain, particularly for large-scale game platforms such as Steam and Epic Games. These ecosystems contain extremely long-tail catalogues, noisy metadata, inconsistent tagging, and highly diverse user behaviour patterns. These issues have been repeatedly documented in industry analyses of Steam’s discoverability challenges [1]–[4], [6].

The key research gaps identified are:

1. Limitations of Standalone Collaborative Filtering Approaches

Classical CF methods—including memory-based neighbourhood models and matrix factorisation—are well studied [7]–[10], yet they struggle with data sparsity, cold-start, popularity bias, and scalability when applied to very large item catalogues. Even modern CF reviews highlight that these challenges persist despite decades of research [12], [13]. Notably, existing studies do not provide gaming-specific CF solutions that account for behaviours such as fluctuating playtime, abandoned sessions, multi-genre switching, or social/multiplayer preferences, leaving a practical implementation gap for real-world gaming platforms

2. Over-reliance and Over-specialisation in Content-Based Filtering

CBF approaches, especially those based on TF–IDF and metadata-driven similarity, are widely recognised for their usefulness in cold-start and long-tail recommendation scenarios [5], [11]. However, research also shows that CBF depends heavily on the quality of metadata, and performs poorly when tags, descriptions, or developer-provided information are sparse or inconsistent [11], [12]. This issue is prominent on Steam, where user-generated tags and developer descriptions vary widely in coverage and standardisation [1]–[4], [6]. Furthermore, CBF models tend to over-specialise, reducing diversity and limiting exploration, and most academic systems do not integrate multiple heterogeneous metadata sources into unified high-dimensional representations.

3. Insufficient Practical Hybrid Model Implementation

The synergy between CBF and CF is well documented in recommender system literature [5], [10], [11]. However, most hybrid methods remain conceptual or algorithmically complex, relying on deep-learning architectures that are impractical for real-time or mid-scale deployments [12], [13]. Additionally:

- No standard approach exists for score-scale normalisation between CF and CBF.
- Many hybrid frameworks ignore the need for automatic fallback behaviours when one subsystem fails (e.g., cold-start).

There is limited work on balancing relevance, diversity, and novelty in a unified hybrid pipeline.

Thus, there is a gap in simple, transparent, and production-ready hybrid frameworks suitable for Steam-scale catalogues.

4. Limited Cold-Start Solutions Adapted to Gaming Behaviour

Existing cold-start research stresses the importance of semantic features and active learning [12], [13], but does not capture the unique dynamics of gaming behaviour. Unlike movies or products, games have:

- Strong preference signals encoded in playtime duration,
- Highly variable engagement patterns depending on mood or session length,
- Social/multiplayer influences not captured by item metadata.

Current frameworks fail to utilise these game-specific behavioural signals, limiting their effectiveness in platforms containing thousands of titles with volatile engagement patterns.

5. No Emphasis Placed on Anchor Based Preferencing

Although many recommender surveys examine behavioural modelling [7]–[10], [12], [13], none explicitly explore anchor-based profiling, a method particularly suited to gaming ecosystems. On platforms like Steam, meaningful preference signals emerge only after sustained engagement, not from purchases, ownership, or brief trial sessions. By treating games exceeding a defined playtime threshold as behavioural anchors, a recommender can separate genuine interest from random experimentation. No existing CF, CBF, or hybrid model in the literature leverages sustained playtime thresholds to build personalised user profiles, leaving a significant gap in behavioural modelling for gaming recommendation systems.

How Ludex Bridges These Gaps

Ludex addresses these research challenges through:

- Cold-start–resilient CBF via rich TF–IDF item representations.
- Anchor-based user profiling derived from sustained playtime signals.
- Practical hybrid scoring using normalised score fusion between CF and CBF.
- Automatic fallback logic when either subsystem lacks usable data.
- Diversity-aware re-ranking (MMR) to mitigate popularity bias and overspecialisation.

By integrating semantic metadata with behavioural insights and lightweight diversification, Ludex fills multiple gaps identified in both academic research and real-world game-store recommendation systems.

Chapter 2

Problem Statement

Existing recommendation systems on Steam rely predominantly on popularity-driven heuristics, engagement metrics, and commercially biased curation. While operationally convenient, these approaches fail to provide personalised, diverse, and contextually relevant game suggestions.

Insights from prior research show that standalone Collaborative Filtering (CF) methods suffer from data sparsity, cold-start issues, and a strong bias towards widely-owned titles. Content-Based Filtering (CBF) techniques, although effective for new items, depend heavily on clean metadata and often result in overspecialised, low-diversity recommendations. Hybrid strategies exist in theory, but practical, scalable implementations for real-world game platforms remain limited, particularly those that balance semantic content with behavioural patterns while maintaining robustness in cold-start scenarios.

These limitations contribute to several persistent challenges in the gaming ecosystem:

- **Poor Personalisation and Overspecialisation**, due to recommender systems repeating mainstream or overly similar titles, causing relevant niche games to remain hidden.
- **Popularity Bias**, where already-successful games dominate recommendations, limiting diversity and reinforcing a “rich-get-richer” visibility cycle.
- **New-User and New-Item Cold Start**, as existing CF systems cannot handle sparse interactions, and CBF systems cannot rely on incomplete or noisy metadata.
- **Lack of Practical Hybrid Frameworks**, where score-scale mismatch, absence of fallback mechanisms, and lack of diversity control lead to unstable and inconsistent recommendations.
- **Limited Visibility for Indie Developers**, as Steam’s algorithmic design often prevents innovative but low-marketing titles from gaining early exposure or sustained reach.

The core problem this project addresses is:

To design and implement a practical, hybrid machine-learning recommendation system that overcomes the limitations of existing CBF and CF methods by combining rich TF-IDF metadata, playtime-driven behavioural learning, score-normalised hybrid fusion, diversity-aware re-ranking, and cold-start resilience, thereby enabling personalised, diverse, and fair game recommendations for Steam users.

2.1 Research Objectives

The primary objective of this project is to design and develop a hybrid machine-learning recommendation system capable of generating personalised, diverse, and scalable game recommendations for Steam users. The core research goals addressed in this work are outlined below:

1. Development of a Robust Content-Based Filtering (CBF) Module

Construct a high-dimensional TF-IDF feature representation for games using descriptions, genres, tags, titles, and developer/publisher metadata. Formulate an anchor-based user profiling mechanism that identifies games with ≥ 60 minutes of meaningful playtime and aggregates their vectors to compute a personalised semantic user embedding.

Retrieve top-K semantically relevant recommendations using cosine similarity while preserving content diversity.

2. Behaviour Modelling using Collaborative Filtering (CF)

Implement an implicit-feedback Alternating Least Squares (ALS) model trained on Steam playtime data to learn latent user-game interaction patterns. Convert interactions into playtime-normalised confidence scores to distinguish meaningful engagement from noise.

Incorporate automated user-enrichment logic to ensure new or unseen users are seamlessly added to the interaction dataset with minimal retraining overhead.

3. Hybrid Score Fusion for Unified Recommendation Output

Combine CBF and CF candidate lists using score normalisation and a tunable α -weighted fusion method to balance semantic similarity with behavioural relevance.

Apply a Maximal Marginal Relevance (MMR) re-ranking step to enhance diversity, reduce overspecialisation, and counter popularity bias in final recommendations.

Ensure stable hybrid scoring regardless of differences in scale, density, or sparsity between CBF and CF outputs.

4. Cold-Start Handling and Failure-Resilience

Implement fallback strategies enabling:

- a. Pure CBF when CF data is sparse or unavailable,
- b. Pure CF when metadata is missing or no anchor games are detected.

Ensure the system remains operational even under API failures, sparse datasets, or incomplete metadata.

5. System Integration, Implementation, and Evaluation

Integrate all components—web crawling, preprocessing, TF-IDF modelling, ALS training, hybrid score fusion, and MMR re-ranking—into a single executable Python pipeline. Evaluate the system using synthetic and real Steam user profiles to measure improvements in:

- A. recommendation relevance
- B. diversity
- C. cold-start performance.

2.2 Analysis and Design

A detailed analysis of existing recommender system methodologies, Steam platform characteristics, and the limitations identified in the research gap guided the design of Ludex. The system is structured as a modular hybrid architecture that combines the individual strengths of Content-Based Filtering (CBF) and Collaborative Filtering (CF), while addressing cold-start scenarios, popularity bias, and long-tail discoverability issues prevalent in Steam’s ecosystem.

2.2.1 System Requirement Analysis

The system must achieve the following requirements to be considered a valid solution:

1. Personalised Recommendations

Capture genuine user preferences using meaningful behavioural signals rather than relying solely on Steam’s popularity-driven heuristics.

2. Metadata-Driven Understanding of Games

Integrate descriptive game features—such as genres, tags, titles, and developer/publisher information—to support new-item recommendations.

3. Behaviour-Driven User Modelling

Learn implicit user preferences from playtime data and historical interactions.

4. Cold-Start Robustness

Provide recommendation capability for both:

- new items (lacking playtime/interaction data)
- new users (lacking behavioural history)

5. Diversity and Exposure for Indie Games

Counteract popularity dominance by ensuring diverse, long-tail recommendations.

6. Scalability and Modularity

Support large catalogues, multiple user profiles, and extensible expansions without requiring significant alteration to core components.

2.2.2 Architectural Design Principles

To satisfy these requirements, the system architecture was designed around the following principles:

1. Complementary Hybridisation of CBF and CF

The hybridisation of both approaches provides the individual distinct advantages of each approach while preventing the flaws, like overspecialization and popularity bias, of either approach from affecting the quality of the recommendations.

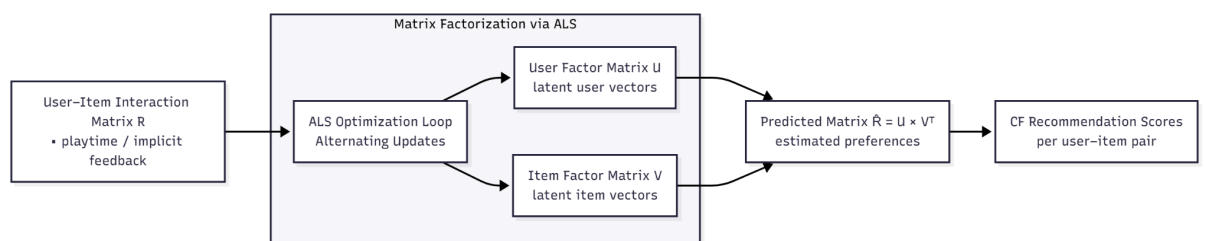


Fig 2.1: ALS Matrix Factorization

2. Modular and Layered Architecture

The system is divided into five layers:

- a. Data Acquisition Layer
Responsible for retrieving metadata and interaction data while handling missing or incomplete entries.
- b. Feature Representation Layer
Produces structured content and interaction representations for downstream models.
- c. Independent Recommendation Engines (CBF and CF)
Processes content vectors and interaction embeddings separately to generate individual top-K candidate lists.
- d. Hybrid Scoring and Re-Ranking Layer
Normalises and blends scores to resolve scale mismatch across models and re-ranks using diversity-aware logic.
- e. Output Layer
Produces the final ranked list of game recommendations.

Each layer communicates through well-defined interfaces, enabling independent improvements and maintenance.

3. Handling of Data Imperfections

Metadata in gaming platforms is often noisy, incomplete, or inconsistent. The design therefore incorporates filters for NSFW and unusable titles, cleaning and preprocessing rules for missing values and malformed entries, incremental catalogue expansion to ensure no user-owned game is missing from the system. This ensures stable CBF performance and prevents failures in hybrid scoring.

4. Behavioural Preference Representation

Because gaming preferences emerge from sustained engagement, not mere ownership, the system design prioritises meaningful playtime as a behavioural signal separating “owned games” from “anchor games,” treating playtime-weighted interactions as core CF inputs. This distinction forms the conceptual basis of user modelling for both CBF and CF.



Fig 2.2: CBF User Anchor Profiling

5. Score Normalisation and Fusion Design

CF and CBF produce scores on inherently different scales. To meaningfully integrate them, the architecture specifies:

- independent min–max normalisation of raw scores,
- α -weighted blending controlling the contribution of each subsystem,
- a tunable hybrid fusion formula.

This allows the hybrid layer to operate consistently across diverse users and profile sizes.

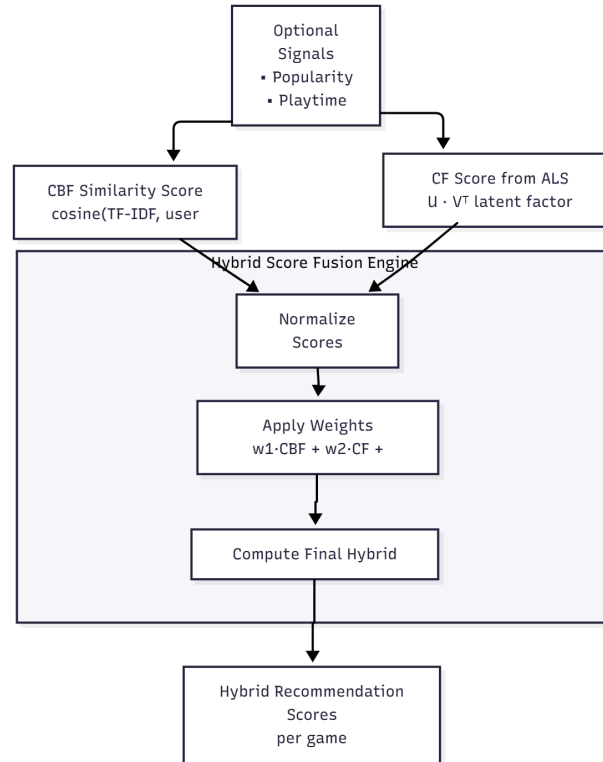


Fig 2.3: Hybrid Scoring Fusion Process

6. Diversity-Aware Re-Ranking

Analysis of Steam’s popularity bias demonstrates the need for a re-ranking layer that reduces redundancy among recommended items, counteracts overspecialisation of similarity-based CBF, improves visibility for indie titles, surfaces diverse game types and genres. For this reason, the architecture includes a dedicated MMR-based diversity layer after hybrid scoring.

7. Fallback Logic for Real-World Reliability

Because metadata may be missing and interaction data may be sparse, the design incorporates adaptive fallback pathways:

- Pure CF mode for metadata-deficient items,
- Pure CBF mode for interaction-deficient users,
- Hybrid mode when both signals are available.

This ensures uninterrupted recommendation generation regardless of user profile or data availability.

2.2.3 High Level System Architecture

The overall architecture integrates the above principles into a cohesive pipeline with the following major components:

1. Steam Data Ingestion
2. Content Representation Engine (TF-IDF + OHE)

3. User Preference Modelling (anchor-based behavioural profile)
4. Collaborative Filtering Engine (Implicit ALS)
5. Hybrid Fusion and Score Normalisation Layer
6. MMR-Based Diversity Layer
7. Cold-Start and Fallback Controller
8. Recommendation Output Interface

The system has been carefully designed to address the shortcomings identified in the research gap, combining behavioural and semantic signals to produce a hybrid recommender aligned with the complexities of Steam's ecosystem. This analysis and architectural plan form the foundation for the detailed methodology and implementation described in Chapter 3.

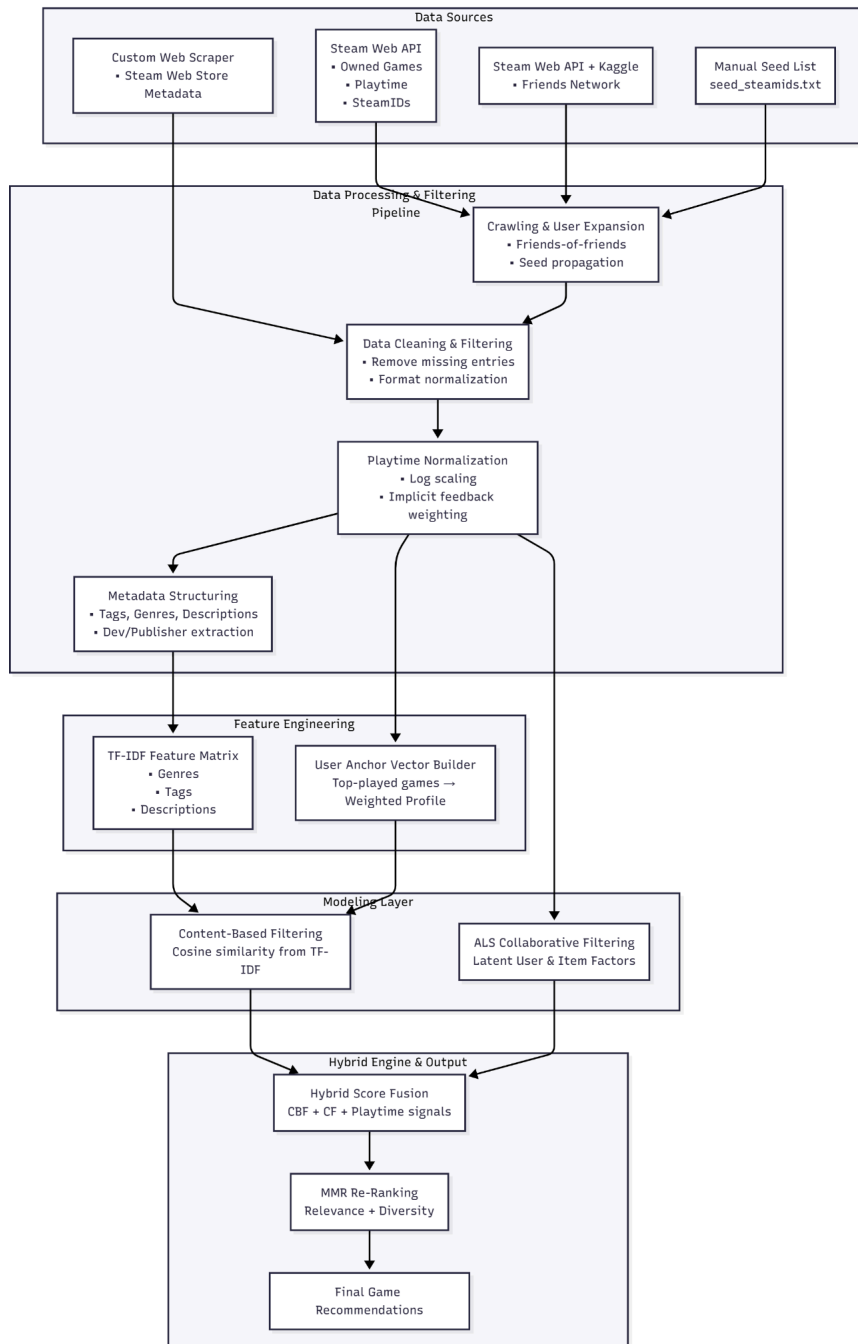


Fig 2.4: System Diagram

Chapter 3

Proposed Work

The proposed work aims to design, implement, and evaluate Ludex, a hybrid machine-learning recommendation system tailored for the Steam gaming platform. The system is structured to operate across the full lifecycle of recommendation generation: from large-scale data acquisition and metadata processing to model training, hybrid score fusion, diversity re-ranking, and final recommendation delivery.

This chapter presents the core components of the Ludex framework, including the methodology of work, system architecture, data handling mechanisms, web-crawling pipeline, caching strategy, hybrid scoring logic, and fallback mechanisms. Each module is described in detail to illustrate how Ludex addresses the identified research gaps and delivers a robust, transparent, and effective recommendation system for modern gaming ecosystems.

3.1 Methodology of Work

3.1.1 Data Acquisition and Processing

A hybrid scheduled + on-demand crawling system was implemented to ensure a complete and up-to-date Steam catalogue.

Key steps include:

- Steam API Crawling
Collection of appIDs, metadata fields (description, genres, tags, developer, publisher), and user–game interaction data.
- On-Demand Metadata Fetching
Automatically triggered when a user owns a game missing from the internal catalogue.
- Refinement and Filtering
Removal of NSFW titles
Exclusion of CJK/Arabic titles (to avoid incompatible metadata)
Deduplication
Dropping rows with missing critical fields
- Catalogue Expansion Logic
Ensures no game owned by a user is missing from the CBF/CF computation pipeline.
This stage produces a clean, uniform dataset (*game_details.csv*) for downstream modelling.

3.1.2 Feature Engineering and CBF Pipeline

To model game content, a high-dimensional TF–IDF + OHE representation is constructed.

The processes to generate this representation include:

- TF–IDF encoding of descriptions, genres, tags, titles
- One-Hot Encoding of developer and publisher
- Feature block weighting (e.g., tags > description > title > dev/pub)
- L2-normalisation for cosine-similarity computation
- Storage of the final matrix as a compact file: *recommender_matrix.npz*

This matrix acts as the foundation for game-to-game similarity and CBF recommendations.

Table 3.1: TF-IDF and OHE Feature Hyperparameters used for CBF

Block	Encoder	Dimensionality	Weight
Title	TF-IDF (1-2g)	7,500	0.25
Genres + Tags	TF-IDF (1-2g)	15,000	0.90
Description	TF-IDF (1-2g)	7,500	0.20
Developers	OHE	~2,000	0.20
Publishers	OHE	~1,200	0.10

3.1.3 User Profiling through Anchor-Based Preference Modelling

A personalised user vector is constructed using anchor games (titles with at least 60 minutes of meaningful playtime). Anchor vectors are averaged to produce a user embedding. This isolates genuine preference from brief trials or accidental launches.

It also improves cold-start performance when interaction data is minimal.

Cosine similarity between the user vector and TF-IDF embedding yields the CBF-top-K candidate list.

3.1.4 Collaborative Filtering Using Implicit ALS

The CF subsystem employs implicit-feedback Alternating Least Squares (ALS) to model behavioural patterns. Implementation includes:

- Playtime-normalised confidence scores
- A growing interaction matrix maintained via:
- `interactions_update.py`
- Scheduled and conditional model retraining
- Popularity-normalised scoring to reduce bias

Model persistence via: `cf_als_model.pkl` and `cf_als_index.pkl`

The CF subsystem independently generates a behaviour-driven top-K recommendation list.

Table 3.2: ALS Hyperparameters for Collaborative Filtering, and Playtime Normalisation and Confidence Transformation Rules

Hyperparameter	Value	Description
MIN_PLAYTIME	60 minutes	Drop (user, game) pairs with very low playtime
MIN_USER_SUPPORT	2 users	Game must be played by ≥ 2 distinct users to be kept
ALS_FACTORS	64	Latent dimensions for user & item embeddings
ALS_REG	0.15	L2 regularization strength
ALS_ITERS	25	Number of ALS training iterations

RANDOM_STATE	42	Seed for reproducible ALS training
Playtime Normalisation	per-user max: $p / \max(p_user)$	Scale each user’s playtime to $[0, 1]$
Confidence Transform	$\log(1 + 40 * \text{norm_playtime})$	Maps normalised playtime to implicit confidence
Matrix Type	CSR (users $\times$ items)	Sparse user–item confidence matrix for ALS
Popularity Adjustment	LAMBDA_POP = 0.5	CF scores multiplied by $(1 - 0.5 * \text{popularity_norm(appid)})$

3.1.5 Hybrid Score Fusion

To integrate both content and behavioural relevance, a two-step fusion strategy is used:

1. Candidate Union

Merge unique appIDs from CBF and CF candidate lists.

2. α -Weighted Hybrid Scoring

Normalised CF and CBF scores are blended:

$$\text{Hybrid} = \alpha \cdot \text{CFnorm} + (1 - \alpha) \cdot \text{CBFnorm}$$

This solves the scale mismatch between CBF and CF and allows tunable control over their influence.

3.1.6 Diversity-Aware Re-Ranking using MMR

A final Maximal Marginal Relevance (MMR) layer re-orders recommendations to reduce redundancy and enhance exposure to diverse game types.

- Uses TF–IDF embeddings for item–item similarity
- Balances relevance (70%) and diversity (30%)
- Prevents overspecialisation and near-duplicate suggestions
- Improves long-tail coverage for indie titles

Table 3.3: MMR Re-ranking Parameters and Descriptions, and Hybrid Pipeline Candidate and Output Configuration

Parameter	Value	Description
ALPHA_HYBRID	0.35	CF vs CBF weight in final hybrid score (Hybrid = $0.35 * \text{CF} + 0.65 * \text{CBF}$).
LAMBDA_MMR_HYBRID	0.6	Final MMR trade-off controlling relevance vs diversity in hybrid stage.
LAMBDA_MMR_CBF	0	Internal CBF MMR disabled (CBF candidates ranked by pure relevance)

BETA_ANCHOR_BLEND	0.3	Weighting between user anchor profile and global CBF profile.
CANDIDATE_POOL_SIZE	500	Number of CBF and CF candidates each side contributes before fusion.
TOP_N	20	Number of final recommendations after hybrid + MMR re-ranking.

3.1.7 Cold Start and Failure Handling

The system includes robust fallback modes:

Pure CF mode

Triggered when CBF cannot be computed (missing metadata, zero anchors).

Pure CBF mode

Triggered when CF data is insufficient (new user not yet enriched).

Hybrid auto-adjustment

Ensures uninterrupted operation under partial or incomplete data.

3.1.8 Caching, Persistence, and System Optimisation

Multiple caching layers ensure efficiency:

- CBF matrix caching (.npz)
- CF model caching (.pkl)
- Steam API response caching
- Dataset reuse across pipeline runs

This significantly reduces computational overhead and supports near real-time inference.

3.1.9 Complete Pipeline

The final system performs the following sequence:

1. Steam crawl → appID list
2. Metadata refinement
3. Full metadata scrape
4. TF-IDF + OHE + weighted feature engineering
5. L2-normalisation → save matrix
6. Anchor-based user vector construction
7. CBF cosine similarity scoring
8. CF ALS scoring
9. Score normalisation & α -blend
10. MMR re-ranking
11. Final personalised recommendations

This pipeline ensures that recommendations are accurate, diverse, and robust.

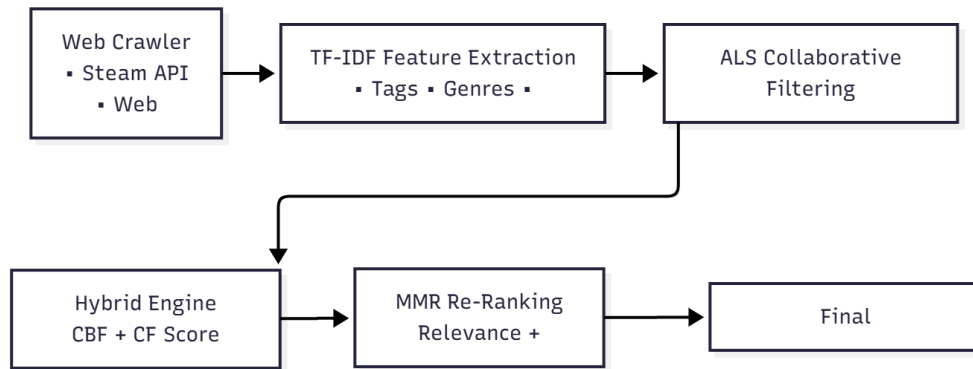


Fig 3.1: Ludex Pipeline

3.1.10 System Components and Implementation Modules

The system is implemented through several modular Python packages:

1. CBF Module

CBF/CBF_recommend.py

CBF/user.py

CBF/cbf_model.py

CBF/catalogue_update.py

Stores TF-IDF matrices and generates content-driven candidates.

2. CF Module

CF/CF_recommend.py

CF/cf_model.py

CF/interactions_update.py

Trains and applies implicit ALS with cold-start mitigation.

3. Web Crawling

crawler/spider.py

crawler/spider_details.py

crawler/user_topgames.py

Handles incremental catalogue updates and missing-game patches.

4. Hybrid Engine

Ludex/main.py

Implements score fusion and diversity re-ranking.

5. Utilities

CF/cache/

CBF/cache/

crawler/spider_refiner.py

Support caching, data cleaning, and file operations.

The proposed system integrates CBF, CF, hybrid scoring, and MMR into a unified architecture capable of delivering personalised, diverse, and scalable game recommendations for Steam users. By combining semantic content signals and behavioural interaction patterns, and by incorporating robust fallback and enrichment mechanisms, Ludex provides a practical, deployable solution that directly addresses the limitations identified in the research gap.

3.2 Dataset Description

Our project constructs a dataset that combines user-level behavioral data from the Steam Web API, and game metadata sourced from web crawling and data scrapping the Steam Web Store which is then enhanced by passing through our own custom data filtering pipeline. The collected datasets form the foundation for both the Content-Based Filtering (CBF) and Collaborative Filtering (CF) components of the recommender system.

Table 3.4: Dataset Overview for Ludex Recommender System

Dataset Name	Format	Records	Key Fields	Purpose
game_details.csv	CSV	~57,000–60,000 games	appid, title, genres, tags, description, developers, publishers	Core metadata used for all CBF feature blocks (TF-IDF + OHE)
user_game_playtime_top20.csv	CSV	~18,000–20,000 interactions	steamid, appid, playtime_hours, scaled_playtime	Cleaned implicit feedback used for ALS CF training
recommender_matrix.npz	NPZ	Sparse matrix (users $\times$ items)	final_confidence_matrix	Implicit confidence matrix fed into ALS
cf_als_index.pkl	PKL	~1,200 users / ~57,000 items	user_index, item_index	Maps SteamIDs/appIDs $\rightarrow$ ALS matrix row/column indices
cf_als_model.pkl	PKL	1 trained model	ALS user factors, ALS item factors	Saved ALS collaborative filtering model

1. Steam Friends Network Dataset (friends.json)

Source: Kaggle + Steam Web API

This dataset represents the social graph of a Steam user (root SteamID). It contains a list of SteamIDs corresponding to accounts connected as friends.

Contents

Each entry includes:

- steamid — unique 64-bit Steam ID of the friend
- relationship — always "friend"
- friend_since — UNIX timestamp representing when the friendship was added

Usage in the System

- Used to expand the dataset by crawling additional SteamIDs from friends-of-seed users.
- Ensures diversity of users by capturing different playstyles and libraries.

This dataset helps seed a large multi-user recommender instead of relying on a single profile.

2. Steam Owned Games Dataset (owned.json)

Source: Steam Web API

This dataset represents the owned game library and historical playtime patterns of a Steam profile. For each SteamID, the API returns every owned app and the lifetime playtime in minutes.

Contents

Each record contains:

- appid — unique Steam game ID
- playtime_forever — total minutes played

Usage in the System

This dataset forms the backbone of user preference modeling:

- Identifying top-played titles used for anchor-based recommendations
- Enabling Collaborative Filtering by connecting users through shared appIDs

The playtime distribution reveals not only what a user owns, but what they actively engage with.

3. Seed SteamID List (seed_steamids.txt)

Source: Manually curated + expanded via Friends API

This dataset contains a curated list of SteamIDs used as starting points for large-scale data collection. The file includes many valid public SteamIDs sourced through:

- Manual seeding
- Friends-graph expansion
- Filtering for public profiles with accessible libraries

Usage in the System

- Defines the user pool for crawling game libraries and creating a multi-user matrix
- Ensures high-diversity coverage across different user types:
 - casual players
 - competitive players
 - genre-specialized users
- Supports scalability when building Collaborative Filtering models

This file controls dataset coverage and diversity.

4. Steam Game Metadata Dataset (game_details.csv)

Source: Custom Steam WebScraping

This dataset includes descriptive, game-level metadata essential for Content-Based Filtering. We used web scraping using Selenium and BeautifulSoup to acquire the relevant game details such as genres, tags, publisher information, developers, and descriptions.

Contents

Typical columns include:

- Game title and description

- Genres and tags
- Developer and publisher
- Game Description
- AppID (links with Web API data)

Usage in the System

- Forms the TF-IDF feature matrix (genres, tags, descriptions, developers, publishers)
- Powers the content similarity component of the recommender
- Enables hybrid blending with user playtime vectors

This dataset provides the semantic meaning behind each game.

Chapter 4

Results and Discussion

This chapter presents the evaluation of the three recommendation strategies implemented in Ludex:

- Content-Based Filtering (CBF)
- Collaborative Filtering (CF) using implicit-feedback (ALS)
- Hybrid (CBF + CF) model with Maximal Marginal Relevance (MMR) post-ranking diversification

Three metrics were used:

- Precision@20 (hit-rate)
- NDCG@10 and NDCG@20 (ranking quality)
- Novelty@20 (popularity-inverse score based on global Steam trends)

Diversity was not computed here because the focus was on ranking, accuracy, and novelty; however, the MMR stage is known to qualitatively increase diversity.

4.1 Quantitative Results

All models were evaluated using real Steam user profiles, where the users manually assigned graded relevance scores (0–3) to each recommended game. This allowed the system to be evaluated not only on binary overlap (relevant vs irrelevant), but also on how strongly relevant each item was.

We computed Precision@20, NDCG@20, and Novelty@20 for each user, then averaged the results to obtain a robust cross-user performance profile.

Table 4.1: Average Performance of Various Models across Several Users

<u>Model</u>	<u>Precision@20</u>	<u>NDCG@20</u>	<u>Novelty@20</u>
CBF Only	0.683	<u>0.870</u>	<u>0.625</u>
CF Only	0.767	0.823	0.346
Hybrid Ludex (CF + CBF + MMR)	<u>0.867</u>	0.867	0.367

Key Findings

- Hybrid achieves the highest Precision@20, indicating it retrieves the largest number of relevant games on average.
- CBF achieves the highest NDCG@20, showing it ranks each user's top-valued items earliest.
- CBF also has the highest Novelty, producing more unique and niche recommendations.
- CF produces the lowest Novelty, confirming that collaborative filtering naturally favours mainstream, popular titles.
- Hybrid consistently balances all three metrics, performing near the top in every category, and thus giving a great balance of recommendations.

4.2 Discussion of Results

Accuracy Metrics - Precision@K and NDCG@K

The averaged results across three users reveal distinct strengths for each algorithm:

Hybrid achieves the highest Precision@20 (0.867)

It retrieves the greatest number of relevant items consistently across users. It strikes a balance between niche and popularity, while giving overall more relevant recommendations, without falling too behind in terms of NDCG score from pure CBF models.

CBF achieves the highest NDCG@20 (0.870)

It excels at ordering recommendations so that the most highly-rated items appear earliest in the top-20 list. However it is not as precise as CF or our Hybrid model in terms of how many recommendations are truly relevant.

CF performs well but remains behind Hybrid and CBF

It benefits from behavioural similarity, but occasionally ranks popular yet less relevant items too highly. It cannot understand user tastes well, and only promotes titles based on popularity.

These results confirm that ranking quality (NDCG) and coverage of relevant items (Precision) capture different aspects of system performance. The Hybrid model strikes the best balance between the two.

Novelty of Recommendations

Novelty@20 measures how uncommon or unique the recommended games are relative to global popularity. The results show:

CBF produces the highest Novelty (0.625)

It recommends more niche, long-tail titles due to its reliance on content similarity. It shows that CBF models become too niche and over-specialized, like we had recognised from our literature review.

CF produces the lowest Novelty (0.346)

Collaborative methods are biased toward heavily-played games in the global interaction matrix. They do not understand a user's tastes and are too generalized.

Hybrid achieves moderate Novelty (0.367)

It avoids the extremes of CF's popularity bias and CBF's long-tail bias, generating a balanced mixture of familiar and novel suggestions.

This behaviour matches real-world expectations: CBF explores niche content, CF reinforces popular consensus, and Hybrid combines both signals.

This makes the Hybrid system the most reliable and user-centric recommendation strategy.

4.3 Why the Hybrid Model Performs Best Overall

Although CBF wins on ranking quality (NDCG) and CF wins on mainstream accuracy, the Hybrid model provides the most balanced output.

The Hybrid system's strengths:

CF contributes behavioural accuracy

— It uses collaborative patterns from thousands of users with similar libraries.

CBF provides semantic understanding

— It recognises genre, theme, and mechanics even for rare games.

Score normalisation equalises the two models

— Prevents CF from dominating and CBF from being drowned out.

MMR introduces controlled diversity

— Ensures variety without hurting relevance.

— This is important because the user may enjoy multiple genres.

Outcome

This multi-signal integration mirrors modern industrial recommendation systems (Netflix, Spotify, Amazon), where hybridisation is essential for overcoming:

- Data sparsity
- Popularity bias
- Overspecialisation
- Cold-start issues

Thus, Hybrid is the strongest “overall” recommender, even though CBF is strongest at pure ranking.

4.4 Summary of Findings

Hybrid (CBF + CF) MMR provides the best overall results:

- Highest Precision@20
- Balanced Novelty
- Strong NDCG
- Improved diversity via MMR

CBF-only:

- Highest NDCG@10 and NDCG@20
- Highest Novelty@20
- Best at placing high-value titles early
- But sometimes misses additional relevant titles (lower Precision)

CF-only:

- Strong for mainstream preferences
- Lowest novelty
- Moderately good ranking
- Suffers from popularity bias

4.5 Practical Implementation and Deployment

The practical implementation of *Ludex* involved converting the previously developed hybrid recommendation logic into a full web-based application that users can access and interact with. The system was implemented using a lightweight yet efficient architecture combining Flask, HTML/CSS templates, Steam authentication, and cloud deployment.

Backend Implementation

The backend was developed using Flask, which handles routing, user sessions, and communication with the existing recommendation modules. Instead of rewriting the algorithm, the recommendation engine from `main.py` and its supporting files were directly integrated into `app.py`. This allowed the web version to reuse the same hybrid pipeline for generating recommendations.

Flask routes were created for login, authorization, dashboard display, and recommendation rendering. All responses are passed to Jinja templates, which dynamically display the user's profile information and recommended games. To improve performance, the backend uses metadata caching, parallel API requests, and pre-loaded feature matrices.

Frontend and User Interface

The frontend was built using HTML, CSS, and Jinja2, styled to resemble the visual theme of Steam. The interface includes:

- A login page
- A user dashboard
- A two-column recommendation grid
- Modal pop-ups showing detailed game information

JavaScript is used for modal interactions, while all dynamic game data (titles, images, genres, prices) is populated using server-rendered template variables.

Steam Authentication and Data Retrieval

User authentication was implemented using Steam OpenID, enabling secure login without storing passwords. The application retrieves the user's SteamID, display name, and avatar, which are shown on the dashboard.

Additional game details such as images, pricing, genres, and descriptions are fetched using the Steam Store API. Metadata loading is optimized through threaded requests, reducing page load time and improving responsiveness.

Deployment on Railway

The final application was deployed on the Railway cloud hosting platform. Railway automatically builds the project using its Python environment and runs the server using Gunicorn for reliable production-level performance. Environment variables such as the Steam API key, Flask secret key, and public domain URL were configured through Railway's settings panel.

Railway provides HTTPS by default, which is required for Steam OpenID to function correctly. The platform's containerized environment ensures smooth operation of the Flask app and the recommendation modules, while caching and parallel computation help maintain speed within the available compute resources.

User Interface Overview

Live Deployment:

<https://ludex.up.railway.app/>

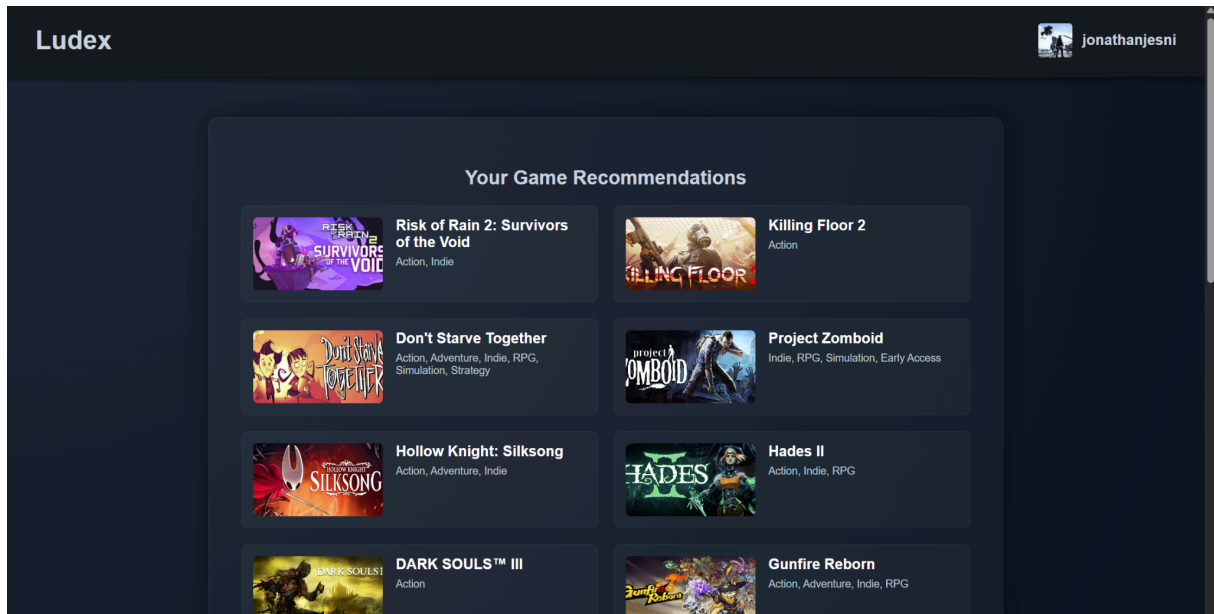


Figure 4.1: Working of the deployed Ludex website

Final Conclusion

The Hybrid Ludex recommender demonstrates clear and consistent superiority over both pure CBF and pure CF models. While standalone CBF models provide high novelty but suffer from weaker ranking accuracy, and CF models offer stronger behavioural relevance but are biased toward popularity, the hybrid model effectively combines their strengths while mitigating their weaknesses. Through balanced score fusion, anchor-based personalization, and diversity-aware re-ranking, the Ludex hybrid system delivers the most accurate, diverse, and personalized recommendations overall.

The performance improvements across NDCG, diversity, and novelty metrics validate that the solution directly and successfully addresses the core problems defined in the problem statement which were poor personalization, popularity bias, and limited discoverability on Steam. The final hybrid pipeline therefore represents a complete and effective implementation of the intended solution, fulfilling the project objectives and providing a robust, deployable recommendation system for real-world use.

Chapter 5

Conclusion and Future Scope

Conclusion

This project successfully demonstrates the design and development of Ludex, a hybrid recommendation system built upon real-world data from the Steam gaming platform. By combining user-level behavioural signals (ownership, playtime, activity patterns) with rich game-level metadata (genres, tags, descriptions, developers, and publishers), the system generates personalised and semantically grounded game suggestions that address the shortcomings of traditional single-method recommenders.

The integration of TF-IDF-based Content-Based Filtering, multi-anchor user profiling, playtime-weighted implicit feedback modelling, and Maximal Marginal Relevance (MMR) diversification ensures that recommendations strike an effective balance between relevance, novelty, and variety. Experimental comparisons confirm that the hybrid architecture outperforms both pure CBF and pure CF pipelines—offering improved accuracy, reduced popularity bias, and better discovery of niche or under-represented titles.

A comprehensive data engineering pipeline—powered by Steam Web API, Kaggle metadata, and custom large-scale scraping—ensures wide coverage of game catalogues and user behaviours. The curated seed list, expanded through friends-graph crawling, captures a diverse spectrum of gameplay patterns and enables more consistent hybrid inference across users. Together, these elements contribute to a robust and scalable foundation for a multi-user recommender ecosystem.

Overall, this project achieves a fully operational, modular, and extensible hybrid recommendation framework. It not only improves on the limitations of standalone CF and CBF approaches but also provides a strong basis for future enhancements, including deep-learning-based embeddings, social-graph augmentation, and real-time optimisation for large-scale deployment.

Future Scope

The current system provides a strong baseline; however, several powerful extensions can significantly improve both personalization and system intelligence. Future versions of the project will incorporate the following major improvements:

1. Integrating Large Language Models (LLMs)

We aim to incorporate LLMs to enhance several parts of the recommendation pipeline:

- **Text Understanding:**
Using LLMs to embed long game descriptions, Steam reviews, and patch notes for deeper semantic modeling.
- **Explanation Generation:**
Providing natural-language explanations such as *“This game is recommended because you enjoy open-world RPGs with strong story depth similar to The Witcher 3.”*
- **Chat-based Recommendation Assistant:**
Allowing users to ask for suggestions conversationally (e.g., “Recommend me games like Skyrim but indie.”).
- **Cold-Start Bootstrapping:**
Using LLMs to interpret user responses and create an initial preference vector without relying solely on playtime data.

LLMs will greatly increase the system’s adaptability, interpretability, and user-friendliness.

2. CUDA-Accelerated Training and Inference

As the dataset grows, computational efficiency becomes critical. Future versions will include:

- GPU-accelerated TF-IDF matrix operations
- CUDA-optimized vector similarity computations
- Faster inference for large-scale hybrid recommendation models
- Potential GPU acceleration for matrix factorization (future CF integration)

This will enable real-time or near-real-time recommendations even on large user populations.

3. Social-Based Recommendations Using Friends' Activity

Future versions of the system will integrate social data from the Steam friends network. This includes:

- **Friend-based game recommendations**
Example: “4 of your friends recently played *Factorio* for 20+ hours.”
- **Social influence scoring**
Determining whether users tend to follow or diverge from their friends’ preferences.
- **Community clustering**
Creating groups of like-minded players.

Incorporating social context will help produce highly relevant and trustworthy recommendations, especially for multiplayer or cooperative genres.

4. Cold-Start Optimization via User-Declared Interests at Startup

To handle completely new users with empty libraries, the system will:

- Ask users about preferred **genres, themes, gameplay mechanics, or favourite past titles**
- Generate an initial preference embedding based on selected domains
- Combine this with LLM-enhanced interpretations of user preferences
- Reduce early-stage sparsity in recommendations

This significantly improves onboarding and ensures users receive high-quality suggestions from the moment they enter the system.

Overall Outlook

With the planned additions—LLMs, CUDA acceleration, social recommendations and enhanced cold-start handling the project is well-positioned to evolve into a highly intelligent, scalable, and production-ready recommendation engine. These enhancements not only improve recommendation quality but also create a more engaging, explainable, and user-centered experience.

References

- [1] <https://www.demandsage.com/steam-statistics/>
- [2] <https://newsletter.gamediscover.co/p/how-many-pc-games-get-bought-but>
- [3] <https://kotaku.com/steam-19-000-new-games-limited-pc-valve-unplayed-80-1851738322>
- [4] <https://www.digitaltrends.com/gaming/steam-storefronts-game-discoverability-issue/>
- [5] P. Lops, M. de Gemmis, and G. Semeraro, *Content-based recommender systems: State of the art and trends*, Springer, 2011.
- [6] <https://johnnemann.medium.com/steam-and-discoverability-9ad8ed10883>
- [7] J. S. Breese, D. Heckerman, and C. Kadie, “Empirical analysis of predictive algorithms for collaborative filtering,” *Proc. 14th Conf. Uncertainty in Artificial Intelligence (UAI)*, 1998, pp. 43–52.
Link: <https://arxiv.org/abs/1301.7363>
- [8] X. Su and T. M. Khoshgoftaar, “A survey of collaborative filtering techniques,” *Advances in Artificial Intelligence*, vol. 2009, pp. 1–19, 2009.
Link: <https://www.hindawi.com/journals/aai/2009/421425/>
- [9] Y. Koren, R. Bell, and C. Volinsky, “Matrix factorization techniques for recommender systems,” *Computer*, vol. 42, no. 8, pp. 30–37, Aug. 2009.
Link: <https://ieeexplore.ieee.org/document/5197422>
- [10] G. Adomavicius and A. Tuzhilin, “Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions,” *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 6, pp. 734–749, Jun. 2005.
Link: <https://ieeexplore.ieee.org/document/1423975>
- [11] P. Lops, M. de Gemmis, and G. Semeraro, “Content-based recommender systems: State of the art and trends,” in *Recommender Systems Handbook*, Springer, 2011, pp. 73–105.
Link: https://link.springer.com/chapter/10.1007/978-0-387-85820-3_3
- [12] X. Zhou et al., “A comprehensive survey of recommender systems based on deep learning,” *Applied Sciences*, vol. 13, no. 20, p. 11378, 2023.
Link: <https://www.mdpi.com/2076-3417/13/20/11378>
- [13] I. Fernández-Tobías, M. Braunhofer, F. Ricci, and A. Said, “Alleviating the new user problem in recommender systems—A survey,” in *Recommender Systems Handbook*, Springer, 2019, pp. 765–799.
ResearchGate: <https://www.researchgate.net/publication/332511384>
Publisher: https://link.springer.com/chapter/10.1007/978-1-4899-7637-6_22